TPO POO - INFORME FINAL

Integrantes del grupo:

TEIXEIRA, Tomas L.U: 1209585

RONCHI, Agustin L.U: 1170164

RAMOS, Federico L.U: 1175595

Docente: LEON MARIA ANGELA

Cuatrimestre: 2 – Año: 2026

Curso: Miercoles Turno Mañana: 07:45 hs - 11:45 hs

UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

El presente documento constituye el informe formal de finalización correspondientes a las tareas de la Etapa 2 del sistema de gestión veterinaria. Se detallan las nuevas funcionalidades implementadas en la arquitectura, la cobertura de pruebas unitarias/de integración, los entregables documentales actualizados y las verificaciones técnicas ejecutadas.

Resumen de la implementación

En esta Etapa 2 logramos convertir el diseño inicial de la Clínica Veterinaria Patitas en una aplicación Java 100% ejecutable. Mantuvimos las jerarquías principales de pacientes, personal y productos, y sumamos controladores para organizar todo el flujo sin mezclar la lógica con el menú visual.

Durante el desarrollo incorporamos funcionalidades clave para que la clínica funcione en el mundo real. Implementamos el registro completo de dueños y de mascotas categorizadas por especie, y sumamos la opción de transferir una mascota a otra familia conservando intacta su historia clínica. Además, creamos un menú administrativo para gestionar el negocio de forma centralizada.

En cuanto al manejo médico, el sistema ahora registra y consume automáticamente los insumos y lotes durante las consultas, y cuenta con alertas para saber cuándo hay poco stock. También configuramos un sistema de alertas para la vacunación: el programa avisa 7 días antes de que toque un refuerzo y detecta automáticamente si una vacuna está vencida tras pasar 30 días.

Finalmente, en la parte de facturación y datos, implementamos la emisión de notas rectificatorias firmadas para consultas ya confirmadas y notas de crédito para poder anular facturas. Todo esto se guarda de forma segura, asegurando la lectura de archivos con el uso explícito de la cláusula finally y guardando detalles importantes como la raza de los pacientes.

Cambios respecto del diseño original

Mientras escribíamos el código nos dimos cuenta de que necesitábamos algunas clases extra de coordinación que no estaban en nuestro UML inicial:

- **Clínica:** Actúa como la raíz del modelo y guarda las colecciones principales.
- **Controladores específicos:** Para no hacer un sistema gigante y desordenado, separamos las tareas en ControladorRegistro, ControladorTurnos, ControladorAtencion, ControladorFacturacion, ControladorAdministracion y ControladorDatos.
- **ListaPrecios:** Decidimos crearla para separar los costos de las entidades principales.
- **RepositorioFacturaMemoria:** Para manejar la persistencia de las facturas.

- **Entidades de reglas de negocio:** Sumamos RecordatorioVacuna, NotaRectificatoria y NotaCredito para cumplir con las reglas estrictas que pedía el enunciado.

Reglas de negocio implementadas

- **Turnos:** Controlamos la duración de las citas, los horarios de los veterinarios, evitamos superposiciones, calculamos recargos por cancelaciones tardías, exigimos seña tras tres ausencias y manejamos urgencias con notificaciones.
- **Historia clínica:** La historia acompaña a la mascota de por vida, aunque cambie de dueño, y solo permite agregar consultas. Si hay un error, no se borra ni se edita, sino que se hace una nota rectificatoria. Además, el sistema bloquea cualquier intento de medicar si no hay un peso registrado.
- **Vacunación:** Validamos la especie y edad, guardamos el número de lote, calculamos los refuerzos automáticamente y avisamos con tiempo de las vacunas próximas o vencidas.
- **Stock:** Controlamos el mínimo, impedimos usar cosas vencidas o que no tienen stock disponible, y configuramos el sistema para que consuma siempre el lote que vence primero.
- **Facturación:** Aplicamos descuentos por efectivo (10%) o plan de salud (15%), dejamos el débito sin recargo y cobramos un 8% extra si pagan con crédito. Una vez cobrada, la factura no se puede alterar; cualquier cambio requiere una nota de crédito.

Conceptos de POO aplicados

- Usamos **herencia y clases abstractas** para estructurar a las mascotas, el personal y los productos.
- Aplicamos **polimorfismo** para manejar las distintas especies de animales y las políticas de pago.
- **Encapsulamos** los atributos y usamos colecciones de Java como List, Map, ArrayList y TreeMap para guardar los datos en memoria.
- Ordenamos datos usando Comparable (para mascotas) y Comparator (para consultas).
- Manejamos errores con excepciones propias como StockInsuficienteException y el uso correcto de try/catch/finally.

Principios SOLID y GRASP

- **SOLID:** Separamos el cálculo matemático de la factura en su propia clase (CalculadorFactura) respetando el Principio de Responsabilidad Única. Además, dejamos el sistema abierto a sumar nuevos métodos de pago a través de la interfaz PoliticaPago (Principio Abierto/Cerrado), y los controladores dependen de abstracciones.

- **GRASP:** Aplicamos el principio de Experto (quien tiene los datos hace el cálculo), Creador (la historia crea las consultas), Controlador (para manejar el menú visual) y Fabricación Pura (para crear los repositorios y listas de precios sin sobrecargar al modelo).

Dificultades encontradas y correcciones

Tuvimos algunos desafíos al juntar el código de todos, especialmente para alinear los diagramas de secuencia con los métodos reales que terminamos programando. Validar el stock sin descontarlo antes de tiempo y manejar varios lotes con distintos vencimientos.

También tuvimos que arreglar el formato de persistencia en los archivos CSV porque nos faltaba guardar la raza de animal de los pacientes. Logramos conectar todas las alertas, registro de clientes y recordatorios que teníamos programados por detrás para que se muestren bien en el menú principal.

Distribución de tareas

Para organizarnos mejor y trabajar en paralelo, dividimos el desarrollo en tres áreas:

- **RONCHI, Agustin (Dominio y modelo core):** Se encargó de las jerarquías, las consultas, las historias clínicas, los productos/lotes, las excepciones y los ordenamientos.
- **RAMOS, Federico (Persistencia y testing):** Armó los repositorios de archivos CSV, el soporte de datos y las pruebas automatizadas.
- **TEIXEIRA, Tomas (Controladores e interfaz):** Manejó el menú por consola, las políticas de pago, la facturación y el flujo general de los casos de uso.

La integración final nos obligó a revisar juntos cómo se comunicaban estas tres partes para ajustar las interfaces comunes.

Pruebas y Verificaciones

Para asegurar que todo funcione perfecto, armamos un plan de pruebas funcionales y sumamos pruebas con JUnit, entre ellas: CalculadorFacturaTest, ControladorTurnosTest, VacunaTest, ConsultaTest y ControladorRegistroTest. También actualizamos el RepositorioMascotaTest para que valide correctamente las razas.

Hicimos validaciones técnicas muy estrictas:

- Todo el código y las pruebas compilan perfectamente con Java 17
- La clase PruebaCompletaDominio ejecuta 21 comprobaciones y todas funcionan sin ningún error.
- Sumamos una prueba de integración que pasó exitosamente por todo el flujo: transferencias, recordatorios, control de stock, rectificaciones y notas de crédito.

Reflexión final

Este trabajo nos sirvió muchísimo para ver cómo se pasa de un texto con reglas de negocio a un modelo UML, y de ahí a una aplicación real y estructurada por responsabilidades. Separar el dominio, los controladores, la persistencia y la interfaz nos permitió trabajar al mismo tiempo sin pisar el código del otro, aplicando todo lo que vimos en la materia.

La lección más grande que nos llevamos es que mantener la coherencia entre los diagramas, las reglas y el código para que a la hora de programar sea más fácil para todos los integrantes ya que trabajamos con los mismo datos.